

Step 1: Load & Explore the Data

First, let's load both CSV files and inspect their structure.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the datasets
pbj_data = pd.read_csv('PBJ_Daily_Nurse_Staffing_Q2_2024.csv',
encoding='latin1') #used encoding='latin1' to avoid
UnicodeDecodeError
nh_provider_info = pd.read_csv('NH_ProviderInfo_Feb2025.csv',
encoding='latin1')

# Display basic info
print(pbj_data.head()) # Check the first few rows
print(nh_provider_info.head()) # Check the structure of the second
dataset
```

C:\Users\CherRyY\AppData\Roaming\Python\Python39\site-packages\IPython\core\interactiveshell.py:3550: DtypeWarning: Columns (0) have mixed types.Specify dtype option on import or set low_memory=False.
exec(code_obj, self.user_global_ns, self.user_ns)

	PROVNUM	PROVNAME	CITY	STATE	COUNTY_NAME \
0	15009	BURNS NURSING HOME, INC.	RUSSELLVILLE	AL	Franklin
1	15009	BURNS NURSING HOME, INC.	RUSSELLVILLE	AL	Franklin
2	15009	BURNS NURSING HOME, INC.	RUSSELLVILLE	AL	Franklin
3	15009	BURNS NURSING HOME, INC.	RUSSELLVILLE	AL	Franklin
4	15009	BURNS NURSING HOME, INC.	RUSSELLVILLE	AL	Franklin

	COUNTY_FIPS	CY_Qtr	WorkDate	MDScensus	Hrs_RNDON	...
Hrs_LPN_ctr \						
0	59	2024Q2	20240401	51	10.77	...
0.0						
1	59	2024Q2	20240402	52	8.43	...
0.0						
2	59	2024Q2	20240403	53	11.13	...
0.0						
3	59	2024Q2	20240404	52	12.27	...
0.0						
4	59	2024Q2	20240405	52	4.95	...
0.0						

	Hrs_CNA	Hrs_CNA_emp	Hrs_CNA_ctr	Hrs_NAtrn	Hrs_NAtrn_emp
Hrs_NAtrn_ctr \					
0	160.08	160.08	0.0	0.0	0.0

0.0					
1	135.95	135.95	0.0	0.0	0.0
0.0					
2	150.31	150.31	0.0	0.0	0.0
0.0					
3	133.01	133.01	0.0	0.0	0.0
0.0					
4	137.92	137.92	0.0	0.0	0.0
0.0					

	Hrs_MedAide	Hrs_MedAide_emp	Hrs_MedAide_ctr
0	0.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0
3	0.0	0.0	0.0
4	0.0	0.0	0.0

[5 rows x 33 columns]

CMS Certification Number (CCN)

Provider Name \

0		015009	BURNS NURSING HOME, INC.
1		015010	COOSA VALLEY HEALTHCARE CENTER
2		015012	HIGHLANDS HEALTH AND REHAB
3		015014	EASTVIEW REHABILITATION & HEALTHCARE CENTER
4		015015	PLANTATION MANOR NURSING HOME

	Provider Address	City/Town	State	ZIP Code \
0	701 MONROE STREET NW	RUSSELLVILLE	AL	35653
1	260 WEST WALNUT STREET	SYLACAUGA	AL	35150
2	380 WOODS COVE ROAD	SCOTTSBORO	AL	35768
3	7755 FOURTH AVENUE SOUTH	BIRMINGHAM	AL	35206
4	6450 OLD TUSCALOOSA HIGHWAY	MC CALLA	AL	35111

	Telephone Number	Provider SSA	County Code	County/Parish \
0	2563324110		290	Franklin
1	2562495604		600	Talladega
2	2562183708		350	Jackson
3	2058330146		360	Jefferson
4	2054776161		360	Jefferson

	Ownership Type	...	\
0	For profit - Corporation	...	
1	For profit - Corporation	...	
2	Government - County	...	
3	For profit - Individual	...	

```

4 For profit - Individual ...

Number of Citations from Infection Control Inspections Number of
Fines \
0 NaN
1
1 0.0
0
2 NaN
0
3 0.0
0
4 NaN
0

Total Amount of Fines in Dollars Number of Payment Denials \
0 23989.0 0
1 0.0 0
2 0.0 0
3 0.0 0
4 0.0 0

Total Number of Penalties
Location \
0 1 701 MONROE STREET
NW,RUSSELLVILLE,AL,35653
1 0 260 WEST WALNUT
STREET,SYLACAUGA,AL,35150
2 0 380 WOODS COVE
ROAD,SCOTTSBORO,AL,35768
3 0 7755 FOURTH AVENUE
SOUTH,BIRMINGHAM,AL,35206
4 0 6450 OLD TUSCALOOSA HIGHWAY,MC
CALLA,AL,35111

Latitude Longitude Geocoding Footnote Processing Date
0 34.5149 -87.736 NaN 2025-02-01
1 33.1637 -86.254 NaN 2025-02-01
2 34.6611 -86.047 NaN 2025-02-01
3 33.5595 -86.722 NaN 2025-02-01
4 33.3222 -87.034 NaN 2025-02-01

[5 rows x 103 columns]

```

Step 2: Data Merging

We need to merge the two datasets based on a common identifier, likely the Provider Number.

```

# Ensure column names are aligned
pbj_data.columns = pbj_data.columns.str.strip().str.lower()

```

```

nh_provider_info.columns =
nh_provider_info.columns.str.strip().str.lower()

# Convert column names to lowercase for consistency
pbj_data.columns = pbj_data.columns.str.lower()
nh_provider_info.columns = nh_provider_info.columns.str.lower()

# Merge datasets on Provider Number (PROVNUM and CMS Certification
Number)
merged_data = pd.merge(pbj_data, nh_provider_info, left_on='provnum',
right_on='cms certification number (ccn)', how='left')

# Check first few rows
print(merged_data.head())

```

	provnum	provname	city	state_x	county_name
0	15009	BURNS NURSING HOME, INC.	RUSSELLVILLE	AL	Franklin
1	15009	BURNS NURSING HOME, INC.	RUSSELLVILLE	AL	Franklin
2	15009	BURNS NURSING HOME, INC.	RUSSELLVILLE	AL	Franklin
3	15009	BURNS NURSING HOME, INC.	RUSSELLVILLE	AL	Franklin
4	15009	BURNS NURSING HOME, INC.	RUSSELLVILLE	AL	Franklin

	county_fips	cy_qtr	workdate	mdscensus	hrs_rndon	...	\
0	59	2024Q2	20240401	51	10.77	...	
1	59	2024Q2	20240402	52	8.43	...	
2	59	2024Q2	20240403	53	11.13	...	
3	59	2024Q2	20240404	52	12.27	...	
4	59	2024Q2	20240405	52	4.95	...	

	number of citations from infection control inspections	number of fines	\
0		NaN	
NaN			
1		NaN	
NaN			
2		NaN	
NaN			
3		NaN	
NaN			
4		NaN	
NaN			

	total amount of fines in dollars	number of payment denials	\
0	NaN	NaN	
1	NaN	NaN	

2	NaN	NaN
3	NaN	NaN
4	NaN	NaN

	total number of penalties	location	latitude	longitude	\
0	NaN	NaN	NaN	NaN	
1	NaN	NaN	NaN	NaN	
2	NaN	NaN	NaN	NaN	
3	NaN	NaN	NaN	NaN	
4	NaN	NaN	NaN	NaN	

	geocoding footnote	processing date
0	NaN	NaN
1	NaN	NaN
2	NaN	NaN
3	NaN	NaN
4	NaN	NaN

[5 rows x 136 columns]

Step 3: Data Cleaning & New Metrics

```
# Fill missing contractor values with 0
merged_data[['hrs_lpn_ctr', 'hrs_cna_ctr']] =
merged_data[['hrs_lpn_ctr', 'hrs_cna_ctr']].fillna(0)

# Calculate total staffing hours (RNs, LPNs, CNAs)
merged_data['total_staffing_hours'] = merged_data['hrs_rndon'] +
merged_data['hrs_lpn'] + merged_data['hrs_cna']

# Calculate contractor percentage
merged_data['contractor_percentage'] = ((merged_data['hrs_lpn_ctr'] +
merged_data['hrs_cna_ctr']) / merged_data['total_staffing_hours']) *
100

# Define "understaffed" (staffing hours < 70% of recommended)
merged_data['understaffed'] = merged_data['total_staffing_hours'] <
(merged_data['mdscensus'] * 0.7)
```

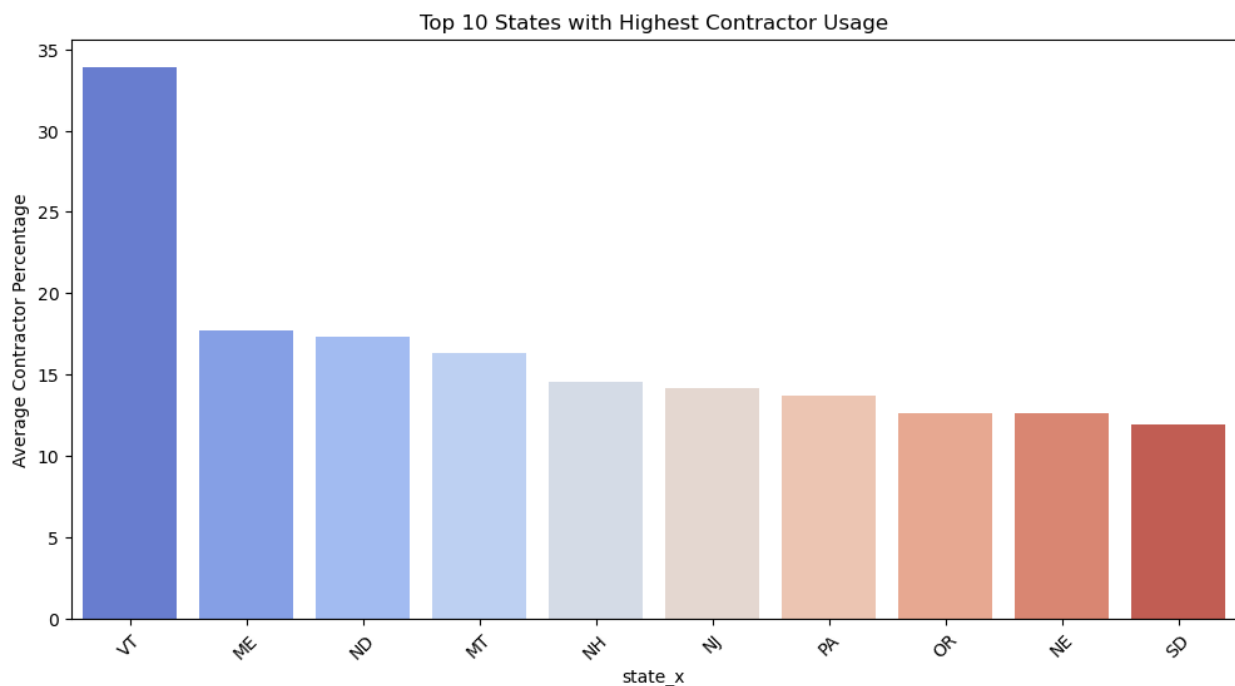
Step 4: Visualizations

1. Bar Chart: Top 10 States with Highest Contractor Nurse Usage

top 10 states with highest contractor usage percentage and sort in descending order of contractor percentage usage

```
# Group by state and calculate average contractor percentage
top_states = merged_data.groupby('state_x')
['contractor_percentage'].mean().sort_values(ascending=False).head(10)
```

```
# Plot
plt.figure(figsize=(12, 6))
sns.barplot(x=top_states.index, y=top_states.values,
palette='coolwarm')
plt.xlabel('state_x')
plt.ylabel('Average Contractor Percentage')
plt.title('Top 10 States with Highest Contractor Usage')
plt.xticks(rotation=45)
plt.show()
```



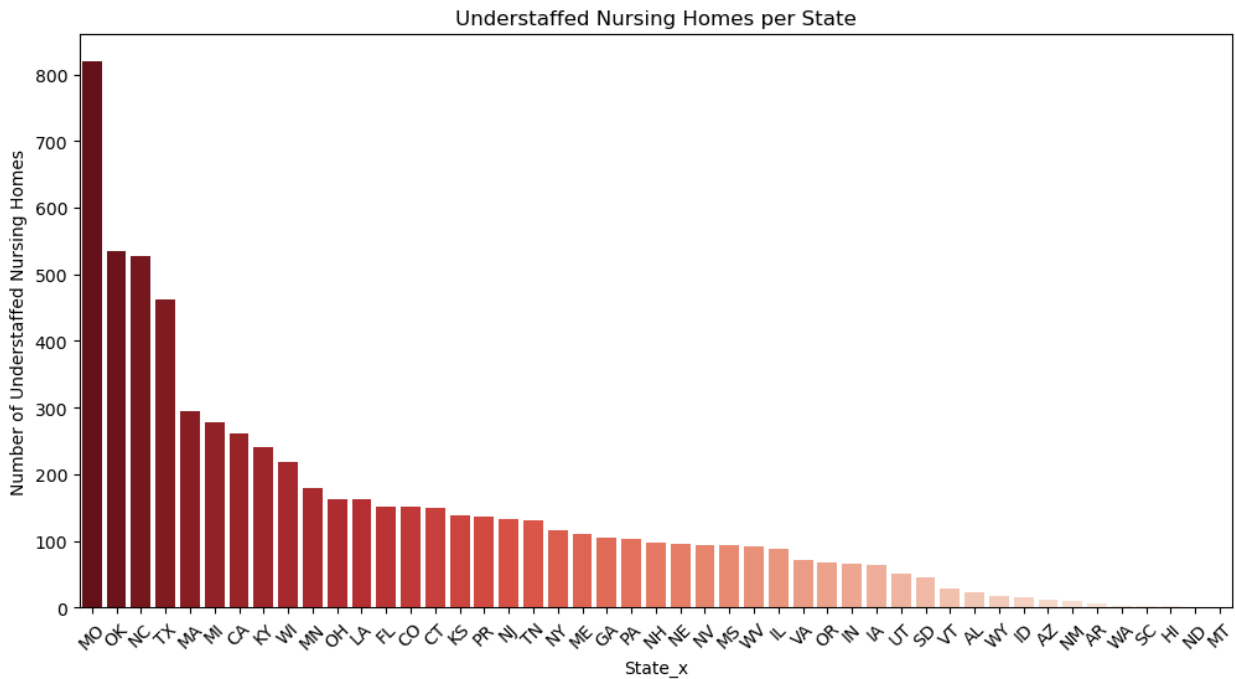
Insight: Clipboard Health should target states with high contractor usage.

2. Bar Chart: Number of Understaffed Nursing Homes per State

```
# Count understaffed nursing homes per state
understaffed_counts =
merged_data[merged_data['understaffed']].groupby('state_x').size().sort_values(ascending=False)

# Plot
plt.figure(figsize=(12, 6))
sns.barplot(x=understaffed_counts.index, y=understaffed_counts.values,
palette='Reds_r')
plt.xlabel('State_x')
plt.ylabel('Number of Understaffed Nursing Homes')
plt.title('Understaffed Nursing Homes per State')
```

```
plt.xticks(rotation=45)
plt.show()
```



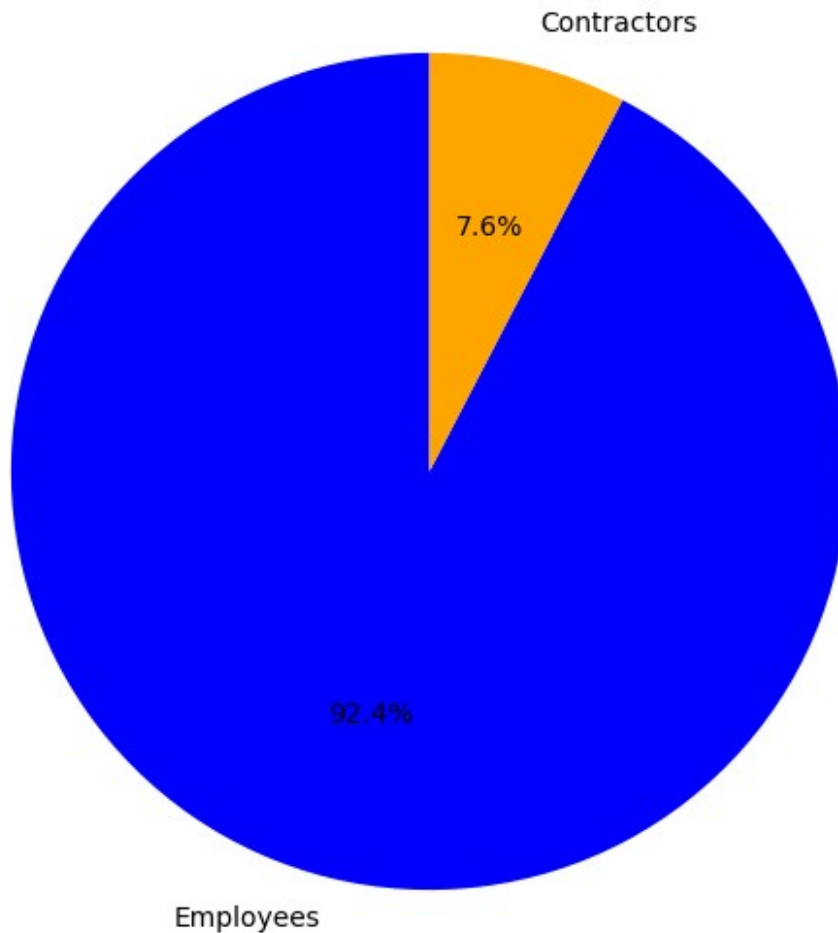
Insight: Clipboard Health should prioritize outreach to states with high understaffing rates.

3 Pie Chart: Employee vs. Contractor Staffing

```
# Calculate total employee and contractor hours
total_employee_hours = merged_data['hrs_cna_emp'].sum() +
merged_data['hrs_lpn_emp'].sum()
total_contractor_hours = merged_data['hrs_cna_ctr'].sum() +
merged_data['hrs_lpn_ctr'].sum()

# Create pie chart
plt.figure(figsize=(7, 7))
plt.pie(
    [total_employee_hours, total_contractor_hours],
    labels=['Employees', 'Contractors'],
    autopct='%1.1f%%',
    colors=['blue', 'orange'],
    startangle=90
)
plt.title("Employee vs. Contractor Staffing in Nursing Homes")
plt.show()
```

Employee vs. Contractor Staffing in Nursing Homes



Insight: If contractor usage is low, Clipboard Health can promote contract staffing solutions.

4 Bar Chart: Nursing Homes with the Most Fines

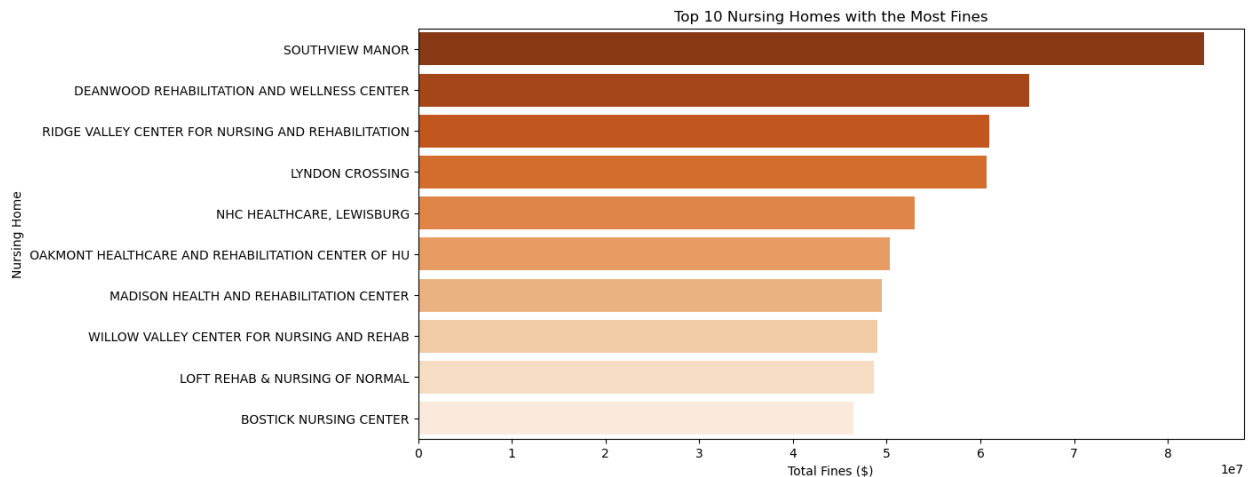
```
# Convert fine amounts to numeric
merged_data['total amount of fines in dollars'] =
pd.to_numeric(merged_data['total amount of fines in dollars'],
errors='coerce')

# Get top 10 nursing homes with highest fines
top_fines = merged_data.groupby('provname')['total amount of fines in
dollars'].sum().sort_values(ascending=False).head(10)

# Plot
plt.figure(figsize=(12, 6))
sns.barplot(x=top_fines.values, y=top_fines.index,
```



```
palette='Oranges_r')
plt.xlabel('Total Fines ($)')
plt.ylabel('Nursing Home')
plt.title('Top 10 Nursing Homes with the Most Fines')
plt.show()
```



Insight: Homes with high fines may have staffing or compliance issues—potential customers for Clipboard Health.

Key Recommendations for Clipboard Health

Here's how Clipboard Health can leverage staffing trends to maximize impact and business growth:

1 Target High Contractor-Usage States

Some states already rely heavily on contract nurses, which means nursing homes in these areas are familiar with the model and more open to staffing partnerships. Clipboard Health can focus marketing and sales efforts here to convert existing demand into business opportunities with minimal resistance.

2 Support Understaffed Nursing Homes

Nursing homes operating with 30% or more staffing shortages are at risk of non-compliance, poor patient care, and regulatory penalties. By proactively reaching out to these facilities, Clipboard Health can position itself as a reliable staffing solution to help them meet minimum requirements and improve patient outcomes.

3 Promote Contractor Staffing to Low-Usage States

Some states still rely almost entirely on permanent staff instead of contract nurses, which may lead to burnout and long-term retention issues. Clipboard Health can introduce the benefits of flexible staffing, educating facilities on how contractors can reduce employee turnover and fill critical gaps without long hiring cycles.

4 Engage Nursing Homes with High Fines

Nursing homes that have received large fines often face issues related to staffing shortages, patient safety, and regulatory compliance. Clipboard Health can position its services as a proactive solution to help these homes avoid future penalties by ensuring they maintain adequate staffing levels.

1. Query to return the customer_name, product_name, and total_amount for each sale in the last 30 days

```
SELECT
    c.customer_name,
    p.product_name,
    s.total_amount
FROM
    Sales s
JOIN
    Customers c ON s.customer_id = c.customer_id
JOIN
    Products p ON s.product_id = p.product_id
WHERE
    s.sale_date >= CURRENT_DATE - INTERVAL 30 DAY;
```

2. Query to find the total revenue generated by each product category in the last year

```
SELECT
    p.category,
    SUM(s.total_amount) AS total_revenue
FROM
    Sales s
JOIN
    Products p ON s.product_id = p.product_id
WHERE
    s.sale_date >= CURRENT_DATE - INTERVAL 1 YEAR
GROUP BY
    p.category;
```

3. Query to return all customers who made purchases in 2023 and are located in the "West" region

```
SELECT DISTINCT
    c.customer_name
FROM
    Sales s
JOIN
    Customers c ON s.customer_id = c.customer_id
WHERE
    YEAR(s.sale_date) = 2023 AND c.sales_region = 'West';
```

4. Query to display the total number of sales, total quantity sold, and total revenue for each customer

```
SELECT
    c.customer_name,
    COUNT(s.sales_id) AS total_sales,
    SUM(s.quantity) AS total_quantity,
    SUM(s.total_amount) AS total_revenue
FROM
    Sales s
```

```
JOIN
  Customers c ON s.customer_id = c.customer_id
GROUP BY
  c.customer_name;
```

5. Query to find the top 3 customers (by total revenue) in the year 2023

```
SELECT
  c.customer_name,
  SUM(s.total_amount) AS total_revenue
FROM
  Sales s
JOIN
  Customers c ON s.customer_id = c.customer_id
WHERE
  YEAR(s.sale_date) = 2023
GROUP BY
  c.customer_name
ORDER BY
  total_revenue DESC
LIMIT 3;
```

6. Query to rank products by their total sales quantity in 2023

```
SELECT
  p.product_name,
  SUM(s.quantity) AS total_quantity_sold,
  RANK() OVER (ORDER BY SUM(s.quantity) DESC) AS rank
FROM
  Sales s
JOIN
  Products p ON s.product_id = p.product_id
WHERE
  YEAR(s.sale_date) = 2023
GROUP BY
  p.product_name;
```

7. Query that categorizes customers into "New" or "Existing" based on their sign_up_date

```
SELECT
  c.customer_name,
  c.sales_region,
  CASE WHEN c.sign_up_date >= CURRENT_DATE - INTERVAL 6 MONTH THEN 'New'
        ELSE 'Existing'
  END AS customer_category
FROM
  Customers c;
```

8. Query to return the month and year along with the total sales for each month for the last 12 months

```
SELECT
```

```

    DATE_FORMAT(s.sale_date, '%Y-%m') AS month_year,
    SUM(s.total_amount) AS total_sales
FROM
    Sales s
WHERE
    s.sale_date >= CURRENT_DATE - INTERVAL 12 MONTH
GROUP BY
    DATE_FORMAT(s.sale_date, '%Y-%m')
ORDER BY
    DATE_FORMAT(s.sale_date, '%Y-%m');

```

9. Query to return product categories that generated more than \$50,000 in revenue during the last 6 months

```

SELECT
    p.category,
    SUM(s.total_amount) AS total_revenue
FROM
    Sales s
JOIN
    Products p ON s.product_id = p.product_id
WHERE
    s.sale_date >= CURRENT_DATE - INTERVAL 6 MONTH
GROUP BY
    p.category
HAVING
    SUM(s.total_amount) > 50000;

```

10. Query to check for any sales where the total_amount doesn't match the expected value (i.e., quantity * price)

```

SELECT
    s.sales_id,
    c.customer_name,
    p.product_name,
    s.quantity * p.price AS expected_total,
    s.total_amount AS actual_total,
    CASE WHEN (s.quantity * p.price != s.total_amount)
    THEN 'Mismatch' ELSE 'Correct' END

```